

Guia Completa

Proyectos 2 y 3 de Robotica

INFO1167 — Alberto Caro

Todo lo que necesitas saber para defender en 2 dias

Explicado como si no supieras nada

Índice

1. Conceptos Basicos	2
1.1. Que es una grilla?	2
1.2. Conceptos clave	2
2. Proyecto 2 — MDP y SMDP	3
2.1. Que es un MDP?	3
2.1.1. Los 5 ingredientes del MDP	3
2.2. La ecuacion de Bellman	3
2.3. Factor de descuento $\gamma = 0,97$	4
2.4. Probabilidad de exito (90 %)	4
2.5. El codigo del MDP	5
2.6. Que es un SMDP?	6
2.6.1. Por que importa el tiempo?	6
2.6.2. Diferencia en el codigo	6
3. Proyecto 3 — Q-Learning SMDP	7
3.1. Que es Q-Learning?	7
3.2. La tabla Q	7
3.3. La ecuacion de actualizacion	7
3.4. Epsilon-greedy (explorar vs explotar)	8
3.5. Alpha (tasa de aprendizaje)	8
3.6. El codigo del Q-Learning	9
4. Diferencias Clave entre los Proyectos	10
5. Cheat Sheet para la Defensa Oral	11
6. Rubricas de Evaluacion	13
6.1. Proyecto 2 (100 puntos)	13
6.2. Proyecto 3 (100 puntos)	13
7. Parametros y Controles	14
7.1. Sliders (ajustables en vivo)	14
7.2. Controles Proyecto 2	14
7.3. Controles Proyecto 3	14
7.4. Como correr	14
8. Apéndice: Formulas Rapidas	15

1. Conceptos Basicos

1.1. Que es una grilla?

Imagina un tablero de ajedrez pero mas chico: **6 filas por 7 columnas**. Cada casilla puede ser:

- **Libre (S)** — el robot puede caminar ahi
- **Muro (X)** — obstaculo, no puede pasar
- **Meta (M)** — el objetivo, donde tiene que llegar
- **Vacio (.)** — no existe, no es parte del mapa

Nuestro mapa se ve asi:

```

      0 1 2 3 4 5 6
0 . S S S S . .
1 . S . . S . .
2 S S S X S S S
3 . . S . S . .
4 . . S S S . .
5 . . . M . . .

```

El robot empieza en una posicion **random** de las casillas **S** y tiene que llegar a **M**. Hay 18 estados transitables y 1 muro.

1.2. Conceptos clave

Estado

Un **estado** es simplemente “donde esta el robot ahora”. Si esta en fila 2, columna 1, su estado es (2, 1). Cada casilla libre es un estado posible.

Accion

Una **accion** es un movimiento: Norte, Sur, Este u Oeste. El robot elige una accion en cada estado.

Politica

La **politica** es la lista de “si estoy aca, voy para alla”. Para cada casilla, dice cual es el mejor movimiento. Como un GPS que siempre te dice para donde doblar.

Algoritmo

Un **algoritmo** es una receta paso a paso. “Primero hace esto, despues hace esto otro, y si pasa esto, hace esto otro.”

2. Proyecto 2 — MDP y SMDP

2.1. Que es un MDP?

MDP = *Markov Decision Process* (Proceso de Decision de Markov).

Imagina que estas en un laberinto oscuro. No puedes ver nada, pero tienes un mapa mental. Cada vez que das un paso:

- Pierdes **1 punto** (costo de caminar)
- Si llegas a la salida, ganas **10 puntos**
- A veces resbalas y no te mueves (**10 %** de las veces)

El MDP te dice: “desde cada casilla, ¿para donde debo ir para ganar la mayor cantidad de puntos?”

2.1.1. Los 5 ingredientes del MDP

Ingrediente	Significado	En nuestro juego
S (Estados)	Todos los lugares posibles	Las 18 casillas libres
A (Acciones)	Lo que puede hacer	Norte, Sur, Este, Oeste
P(s' s,a)	Probabilidad de transicion	90 % exito, 10 % falla
R(s,a)	Recompensa	−1 por paso, +10 en meta
γ	Factor de descuento	0.97

2.2. La ecuacion de Bellman

Esta es la ecuacion **mas importante** del MDP:

$$V(s) = \max_a \left[R(s, a) + \gamma \sum_{s'} P(s'|s, a) \cdot V(s') \right] \quad (1)$$

Desglose termino por termino:

Punto Clave

- $V(s)$ = “que tan bueno es estar en esta casilla”
- $\max_a$ = “elijo la mejor accion de todas”
- $R(s, a)$ = “lo que gano/pierdo ahora” (−1 por caminar)
- $\gamma = 0,97$ = “cuanto me importa el futuro”
- $V(s')$ = “que tan bueno es el lugar donde voy a llegar”

La ecuacion dice: “**el valor de donde estoy = lo que gano ahora + lo que espero ganar en el futuro, eligiendo la mejor accion**”

2.3. Factor de descuento $\gamma = 0,97$

¿Por que 0.97 y no 1.0? Porque si $\gamma = 1,0$, el robot valdria igual una recompensa hoy que en 1000 pasos. Con $\gamma = 0,97$:

Momento	Valor
Ahora	1,0
En 1 paso	0,97
En 2 pasos	$0,97^2 = 0,94$
En 10 pasos	$0,97^{10} = 0,74$

El robot prefiere recompensas cercanas pero no ignora las lejanas.

2.4. Probabilidad de exito (90 %)

Cuando el robot decide ir al Sur:

- **90 %** de las veces: se mueve al Sur correctamente
- **10 %** de las veces: se queda donde esta (fallo)

```
valor = 0.9 * (recompensa + gamma * V[siguiente]) # exito  
        + 0.1 * (recompensa + gamma * V[mismo_lugar]) # fallo
```

2.5. El codigo del MDP

La funcion clave del proyecto 2:

```
def calcular_politica(gamma, modo='MDP'):
    V = {s: 0.0 for s in estados} # todos empiezan en 0
    for _ in range(500): # iterar hasta converger
        for s in estados:
            if s == meta:
                V[meta] = 10.0 # la meta vale 10
                continue
            mejor = float('-inf')
            for acc in ACTIONS: # probar las 4 direcciones
                sig = move(s, acc) # donde llegaria
                r = -1.0 + (10.0 if sig == meta else 0.0)

                # AQUI esta la diferencia MDP vs SMDP
                if modo == 'SMDP':
                    desc = promedio(gamma ** tau) # temporal
                else:
                    desc = gamma # fijo

                val = 0.9*(r + desc*V[sig]) + 0.1*(r + desc*V[s])
                mejor = max(mejor, val)
            V[s] = mejor

    # extraer politica: mejor accion para cada estado
    for s in estados:
        politica[s] = accion_con_mayor_valor
```

Paso a paso:

1. Empezamos con $V(s) = 0$ para todas las casillas
2. Para cada casilla, probamos las 4 acciones
3. Para cada accion, calculamos cuanto vale usando Bellman
4. Nos quedamos con la mejor accion
5. Repetimos hasta que los valores se estabilicen (convergencia)
6. Al final, la politica es: para cada casilla, la accion con mayor valor

2.6. Que es un SMDP?

SMDP = *Semi-Markov Decision Process*.

La unica diferencia con el MDP es que las acciones toman **tiempo variable**:

Accion	Tiempo
Norte	Normal($\mu = 2, \sigma = 0,2$)
Sur	Normal($\mu = 2, \sigma = 0,2$)
Este	Normal($\mu = 3, \sigma = 0,3$)
Oeste	Normal($\mu = 3, \sigma = 0,3$)

2.6.1. Por que importa el tiempo?

Porque el factor de descuento penaliza el tiempo. Si una accion toma mas tiempo:

```
gamma^tau = 0.97^3 = 0.91 (accion lenta, descuenta mucho)
gamma^tau = 0.97^2 = 0.94 (accion rapida, descuenta poco)
```

Punto Clave

El SMDP prefiere acciones rapidas cuando ambas llevan a buen lugar. Norte/Sur son mas rapidos (2s) que Este/Oeste (3s), entonces si ambas opciones llegan a la meta, el SMDP prefiere Norte/Sur.

2.6.2. Diferencia en el codigo

```
# MDP: descuento fijo
desc = gamma # siempre 0.97

# SMDP: descuento variable por tiempo
mu, sigma = TIEMPO_ACCION[accion] # ej: Norte = (2, 0.2)
tau = muestra_de_Normal(mu, sigma) # ej: 1.87 segundos
desc = gamma ** tau # 0.97^1.87 = 0.944
```

3. Proyecto 3 — Q-Learning SMDP

3.1. Que es Q-Learning?

Q-Learning es aprender por prueba y error.

Imagina un raton en un laberinto. Al principio no sabe nada — no sabe donde esta el queso, no sabe donde estan los muros. Solo puede probar caminos.

Despues de muchos intentos, empieza a aprender: “si estoy en esta esquina y voy a la derecha, llego al queso”. Eso es Q-Learning.

3.2. La tabla Q

El robot tiene una tabla donde apunta notas:

```
Q[(2,1), Norte] = -3.2 (si estoy en (2,1) y voy Norte, fue malo)
Q[(2,1), Sur] = 5.1 (si voy Sur, fue bueno!)
Q[(2,1), Este] = -1.0 (neutro)
Q[(2,1), Oeste] = -2.5 (malo)
```

Al principio **todos los valores son 0** (no sabe nada). Despues de muchos episodios, los valores se llenan y el robot sabe que hacer.

3.3. La ecuacion de actualizacion

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[R + \gamma^T \cdot \max_{a'} Q(s', a') - Q(s, a) \right] \quad (2)$$

Desglose

- $Q(s, a)$ = mi nota actual para esta accion en este estado
- $\alpha = 0,10$ = cuanto ajusto (10 % = poco a poco)
- R = lo que acaba de pasar (-1 o $+10$)
- γ^T = descuento por tiempo (igual que SMDP)
- $\max Q(s')$ = lo mejor que puedo esperar del siguiente estado
- $Q(s, a)$ viejo = mi nota antes de esta experiencia

La diferencia entre “lo que esperaba” y “lo que paso” se llama **error temporal**. Si el error es grande, ajusto mucho. Si es chico, ajusto poco.

3.4. Epsilon-greedy (explorar vs explotar)

El robot tiene un dilema:

- **Explotar:** usar lo que ya sabe (ir por el mejor camino conocido)
- **Explorar:** probar algo nuevo (ir por un camino random)

Epsilon controla esto:

```
def elegir(estado, epsilon):  
    if random() < epsilon: # ej: 30% de las veces  
        return accion_aleatoria # EXPLORAR: probar algo nuevo  
    else: # ej: 70% de las veces  
        return mejor_accion(estado) # EXPLOTAR
```

Punto Clave

Al principio epsilon es alto (explorar mucho). Con el tiempo baja (explotar lo aprendido). El slider de epsilon te deja controlar esto en vivo.

3.5. Alpha (tasa de aprendizaje)

$\alpha = 0,10$ significa que cada actualizacion solo cambia el valor en un 10 %:

```
Q_viejo = 5.0  
nueva_info = 7.0  
alpha = 0.10  
  
Q_nuevo = 5.0 + 0.10 * (7.0 - 5.0) = 5.0 + 0.2 = 5.2
```

Si α fuera 1.0, el nuevo valor seria 7.0 (borraria todo lo anterior). Con 0.10, cambia solo un poquito. Esto hace el aprendizaje **estable**.

3.6. El codigo del Q-Learning

```
class AgenteQLearning:
    def __init__(self):
        self.q = defaultdict(float) # todo empieza en 0

    def entrenar_episodio(self, gamma, alpha, epsilon):
        s = get_random_start() # empezar en random

        for _ in range(200): # max 200 pasos
            a = self.elegir(s, epsilon) # epsilon-greedy
            sig = self.mover(s, a) # 90% exito
            r = -1 + (10 if sig == meta else 0)
            tau = self.tiempo_accion(a) # muestrear tiempo
            desc = gamma ** tau # descuento SMDP

            viejo = self.q[(s, a)]
            self.q[(s, a)] = viejo + alpha * (
                r + desc * self.max_q(sig) - viejo
            )

            s = sig
            if s == meta:
                break
```

Paso a paso:

1. El robot empieza en una posicion random
2. Elige una accion (epsilon-greedy)
3. Se mueve (90 % exito, 10 % falla)
4. Recibe recompensa (−1 por paso, +10 si llego a la meta)
5. Mide cuanto tiempo tomo la accion
6. Actualiza la tabla Q
7. Repite hasta llegar a la meta o 200 pasos
8. Repite todo desde paso 1 con otro inicio random

Despues de ~1500 episodios, la tabla Q converge y el robot sabe que hacer en cada situacion.

4. Diferencias Clave entre los Proyectos

Aspecto	Proyecto 2 (MDP/SMDP)	Proyecto 3 (Q-Learning)
Tipo	Basado en modelo	Libre de modelo
Conoce reglas?	SI	NO
Como aprende?	Calcula matematica	Prueba y error
Necesita modelo?	SI (P, R conocidos)	NO (solo experiencia)
Convergencia	Exacta, pocas iter	Aprox, miles episodios
Descuento	γ o γ^T	γ^T siempre
Resultado	$V(s)$ y politica	Tabla $Q(s, a)$ y politica

Punto Clave

Pregunta clave de defensa: “¿Cual es la diferencia entre MDP y Q-Learning?”

Respuesta: MDP necesita conocer las reglas del juego (las probabilidades y recompensas). Q-Learning no necesita nada — aprende jugando. Es como la diferencia entre estudiar el mapa antes de caminar vs aprender caminando.

5. Cheat Sheet para la Defensa Oral

1. Que es un MDP?

Un modelo para tomar decisiones secuenciales bajo incertidumbre. 5 componentes: estados, acciones, probabilidades de transición, recompensas y factor de descuento. La ecuación de Bellman calcula el valor de cada estado y la mejor acción.

2. Que es un SMDP y como se diferencia de un MDP?

Un SMDP extiende el MDP al considerar que las acciones toman tiempo variable. Norte/Sur = $N(2, 0.2)$, Este/Oeste = $N(3, 0.3)$. El descuento cambia de γ a γ^τ , donde τ es el tiempo muestreado.

3. Por que γ^τ ?

Porque si una acción toma más tiempo, la recompensa futura llega más tarde y vale menos. τ grande = descuento grande = la recompensa futura vale menos.

4. Que es Q-Learning?

Algoritmo libre de modelo. El agente mantiene una tabla Q que estima el valor de cada par (estado, acción). Aprende por experiencia: elige acciones, observa recompensas, actualiza Q con: $Q(s, a) + = \alpha \cdot [R + \gamma^\tau \cdot \max_{s'} Q(s') - Q(s, a)]$.

5. Que es alpha?

Tasa de aprendizaje. $\alpha = 0,10$ = ajusta 10 % cada vez. Hace el aprendizaje gradual y estable — no borra todo lo que aprendió antes por una experiencia rara.

6. Que es epsilon-greedy?

Estrategia de exploración vs explotación. Con probabilidad ϵ , el agente elige acción aleatoria (explorar). Con probabilidad $1 - \epsilon$, elige la mejor conocida (explotar).

7. Por que Q-Learning es “libre de modelo”?

No necesita conocer $P(s'|s, a)$ ni $R(s, a)$. Solo necesita interactuar con el entorno y observar que pasa. MDP necesita modelo completo; Q-Learning solo experiencia.

8. Cuantos episodios para converger?

~1500–2000 episodios. La recompensa promedio sube de -17 (al inicio) a $+1$ (cuando converge).

9. Por que los valores SMDP < MDP?

Porque el tiempo extra en E/O (3s vs 1s) hace que γ^τ sea menor que γ . Descuento más agresivo = valores menores.

10. Por que inicio aleatorio?

Para que funcione desde cualquier estado. Q-Learning necesita explorar desde distintos puntos para llenar bien la tabla Q.

6. Rubricas de Evaluacion

6.1. Proyecto 2 (100 puntos)

Criterio	Pts	Que necesitas
Simulacion grafica	20	Pygame, mapa 2D, robot, meta, muros
MDP clasico	30	Value Iteration, $\gamma = 0,97$, $P=90\%$, 4 acciones
SMDP	30	γ^r , $N(2,0.2)$, $N(3,0.3)$, 90 % exito
Defensa oral	20	Explicar diferencia MDP/SMDP, dominar codigo

6.2. Proyecto 3 (100 puntos)

Criterio	Pts	Que necesitas
Simulacion grafica	20	Pygame, mapa 2D, convergencia visible
Q-Learning SMDP	40	Tabla Q, $\alpha = 0,10$, γ^r , epsilon-greedy
Modelado estocastico	20	90 % exito, $N(2,0.2)$, $N(3,0.3)$
Defensa oral	20	Explicar Q-Learning vs MDP, dominar codigo

7. Parametros y Controles

7.1. Sliders (ajustables en vivo)

Slider	Proy 2	Proy 3	Que hace
Gamma	Si	Si	Factor de descuento (0.5 a 0.99)
Alpha	No	Si	Tasa de aprendizaje (0.01 a 0.50)
Epsilon	No	Si	Exploracion (0.01 a 1.0)
Velocidad	Si	Si	Ms entre pasos (100 a 800)

7.2. Controles Proyecto 2

1 o boton MDP Cambia a modo MDP clasico
2 o boton SMDP Cambia a modo SMDP
ESPACIO Pausar / Reanudar
R o boton Reset Reiniciar

7.3. Controles Proyecto 3

ENTER o boton Entrenar 500 episodios
T o boton Auto Hasta convergencia
A o boton Animar Ver robot navegar
R o boton Reset Reiniciar todo

7.4. Como correr

```

pip install pygame numpy
cd C:\Projects\robotica
python proyecto2.py
python proyecto3.py
  
```

8. Apéndice: Formulas Rápidas

Concepto	Formula
Bellman (MDP)	$V(s) = \max_a [R + \gamma \cdot V(s')]$
Bellman (SMDP)	$V(s) = \max_a [R + \mathbb{E}[\gamma^\tau] \cdot V(s')]$
Q-Learning	$Q(s, a) \leftarrow \alpha [R + \gamma^\tau \max Q(s') - Q(s, a)]$
Descuento temporal	$\gamma^\tau = 0,97^\tau$
Transición (90 %)	$0,9 \cdot V(s') + 0,1 \cdot V(s)$
Recompensa	$R = -1$ (paso), $R = +10$ (meta)